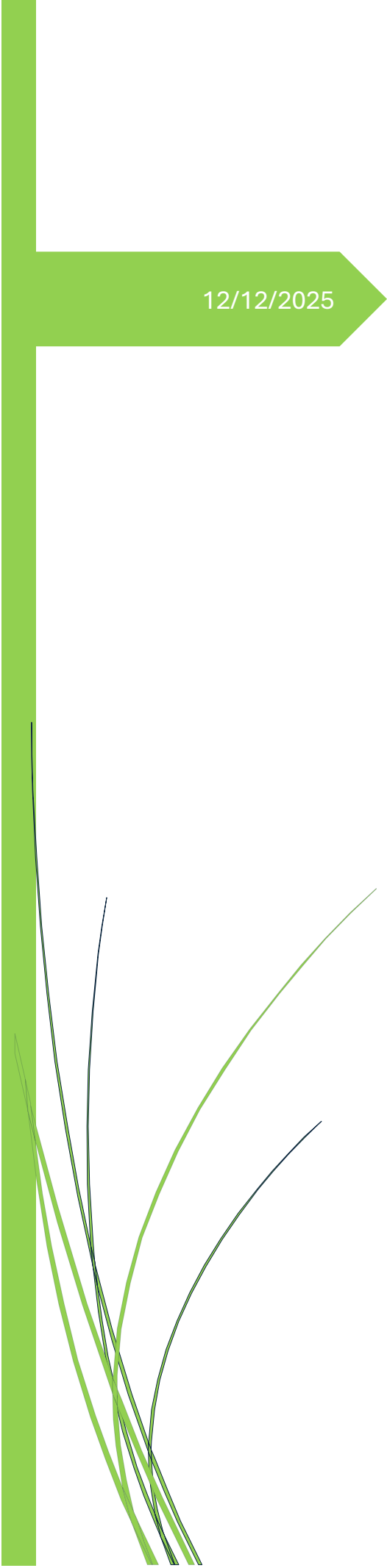
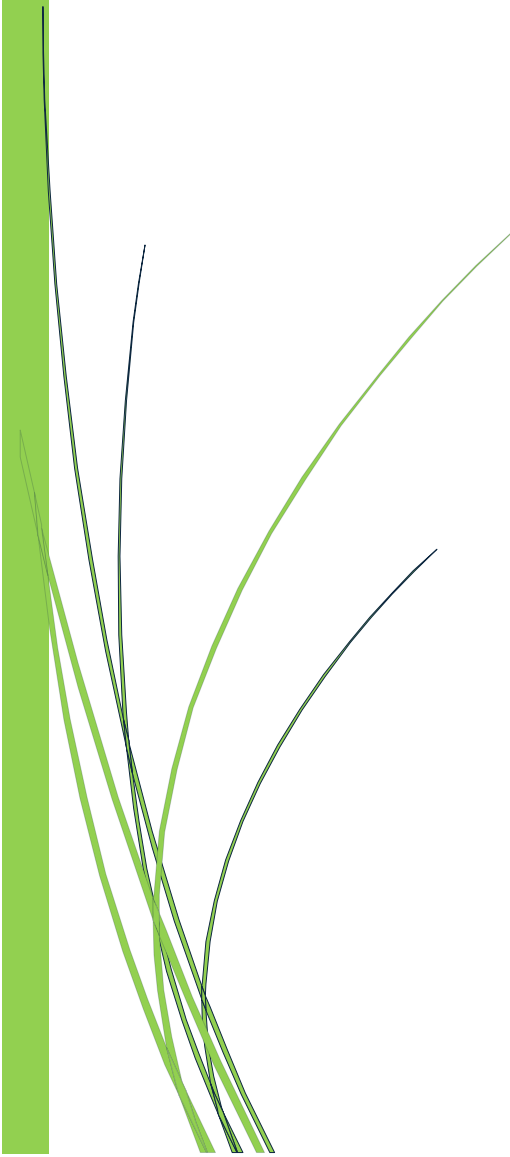


A thick green vertical bar runs down the left side of the page. A green arrow points to the right from the bar, containing the date 12/12/2025.

12/12/2025

# Déploiement d'applications - Projet final

Abstract green and blue lines of varying thicknesses curve upwards from the bottom left corner of the page.

Boualem El Guendouz  
CÉGEP DE CHICOUTIMI

## **Table des matières**

|                                                              |   |
|--------------------------------------------------------------|---|
| Contexte du projet .....                                     | 2 |
| Architecture logique et choix technologiques .....           | 2 |
| Bloc Web / Authentification .....                            | 2 |
| Bloc Base de données d'authentification (PostgreSQL) .....   | 2 |
| Bloc Licences / Application native (.NET) et Azure SQL ..... | 3 |
| Déploiement avec Docker et Docker Compose.....               | 3 |
| Marche à suivre pour reproduire le déploiement .....         | 4 |
| Préparation du code et de la structure des dossiers .....    | 4 |
| Configuration des connexions .....                           | 4 |
| Construction et lancement des services conteneurisés .....   | 4 |
| Initialisation et tests .....                                | 5 |
| Arrêt et nettoyage .....                                     | 5 |
| Conclusion .....                                             | 5 |

## **Contexte du projet**

Le projet consistait à déployer une solution complète composée de plusieurs composants interdépendants. D'un côté, une application web en Node.js/Express permet l'authentification des utilisateurs et leur donne accès au téléchargement d'un installateur Windows. De l'autre, un service d'authentification en Flask interroge une base PostgreSQL pour valider les identifiants. Enfin, une application native .NET gère des licences logicielles à partir d'une base de données relationnelle, initialement prévue en MySQL mais finalement déployée sur Azure SQL (SQL Server). L'objectif était de rendre ce déploiement reproductible, isolé et documenté, en respectant la contrainte de séparer les « serveurs » (front, auth, bases de données).

## **Architecture logique et choix technologiques**

L'architecture est organisée en trois blocs principaux :

### **Bloc Web / Authentification**

Ce bloc comprend l'application web et le service d'authentification, tous deux conteneurisés avec Docker et orchestrés via un fichier de composition. L'application web est développée avec Node.js et Express. Elle expose une page de login, envoie les identifiants à un service d'authentification et, en cas de succès, permet à l'utilisateur de télécharger un exécutable (setupLicencesBoualem.exe) inclus dans l'image Docker. Le choix de Node.js/Express se justifie par sa simplicité pour créer rapidement une API web légère et par son intégration naturelle avec l'écosystème JavaScript (Axios pour les appels HTTP, gestion facile des sessions côté serveur).

Le service d'authentification est une API REST développée en Python avec Flask et SQLAlchemy. Il expose un endpoint POST /auth qui reçoit un nom d'utilisateur et un mot de passe, puis vérifie ces informations dans une table users stockée dans PostgreSQL. L'utilisation de Flask permet de garder un code très simple et lisible, tout en profitant de SQLAlchemy pour rester indépendant du SGBD sous-jacent. Le choix de séparer ce service du front respecte la consigne de ne pas tout centraliser sur le même serveur et suit une approche de type micro-services.

### **Bloc Base de données d'authentification (PostgreSQL)**

La base d'authentification est un PostgreSQL 15 exécuté dans un conteneur dédié. L'initialisation est automatisée via un script init.sql monté dans le conteneur au démarrage, qui crée la base authdb et la table users, puis y insère des comptes de test. Ce choix répond à plusieurs objectifs : isoler physiquement la base du service Flask, faciliter la réinitialisation pour les tests (il suffit de recréer le conteneur), et éviter

d'installer PostgreSQL directement sur la machine de développement. L'URL de connexion est injectée dans l'API Flask via une variable d'environnement (DATABASE\_URL), ce qui respecte la contrainte du professeur de ne modifier que les informations de connexion dans le code existant.

## Bloc Licences / Application native (.NET) et Azure SQL

La gestion des licences est prise en charge par une application native écrite en .NET. Initialement, le professeur proposait un script MySQL pour créer la base et la table Licences. Plutôt que d'héberger MySQL localement en conteneur, la base a été déployée sur Azure SQL (SQL Server) à l'aide d'un script T-SQL adapté. Ce script crée la base, la table Licences et insère l'ensemble des lignes de test (logiciels, type de licence, dates d'expiration, nombre d'utilisateurs).

Pour connecter l'application native à Azure SQL, la configuration appsettings.json a été modifiée afin d'y renseigner la chaîne de connexion du serveur Azure (nom du serveur, base, utilisateur, mot de passe, port 1433 et éventuellement chiffrement). Dans le code C#, le type de connexion utilisé a été ajusté en conséquence : la classe MySqlConnection, propre au fournisseur MySQL, a été remplacée par SqlConnection, le fournisseur standard pour SQL Server. Cette modification reste dans le cadre autorisé par la consigne, puisqu'il s'agit d'adapter les informations de connexion et le provider pour le SGBD réellement utilisé, sans changer la logique métier de l'application.

## Déploiement avec Docker et Docker Compose

Le déploiement du bloc Web/Auth/PostgreSQL repose sur un fichier docker-compose.yml placé à la racine du projet. Ce fichier décrit trois services :

- postgres-db : un conteneur basé sur l'image officielle PostgreSQL. Il expose le port 5432, reçoit le script auth/init.sql pour l'initialisation de la base et est rattaché au réseau Docker app-net.
- auth-service : construit à partir du dossier auth via un Dockerfile Python. Il dépend de postgres-db, récupère l'URL de connexion via la variable DATABASE\_URL et publie le port 5000 sur la machine hôte. Dans le code, SQLAlchemy utilise directement cette URL pour se connecter à la base.
- web-app : construit à partir du dossier web via un Dockerfile Node.js. Il dépend de auth-service, publie le port 3000, et inclut dans son image le dossier files contenant l'installateur Windows. L'application Express contacte l'API d'authentification en utilisant le nom de service Docker (http://auth-service:5000/auth), ce qui fonctionne grâce au réseau partagé app-net.

L'utilisation de Docker Compose permet de démarrer l'ensemble avec une seule commande (`docker compose up -d --build`), de documenter explicitement les dépendances entre services (`depends_on`) et de conserver une topologie cohérente sur n'importe quelle machine où Docker est installé.

## **Marche à suivre pour reproduire le déploiement**

Pour reproduire le système sur une nouvelle machine, la démarche se déroule en plusieurs étapes structurées.

### **Préparation du code et de la structure des dossiers**

Placer le projet dans un dossier racine contenant :

- `auth/` avec `app.py`, `requirements.txt`, `init.sql` et un Dockerfile Python.
- `web/` avec `app.js`, `package.json`, un dossier `files/` contenant `setupLicencesBoualem.exe` et un Dockerfile Node.js.
- `docker-compose.yml` décrivant les services `postgres-db`, `auth-service` et `web-app`.

### **Configuration des connexions**

- Dans `auth/app.py`, s'assurer que la chaîne de connexion PostgreSQL est récupérée via `DATABASE_URL` (soit depuis l'environnement, soit avec une valeur par défaut pointant sur `postgres-db:5432`).
- Dans `web/app.js`, vérifier que l'appel Axios du login cible bien `http://auth-service:5000/auth` pour profiter de la résolution de noms dans le réseau Docker.
- Dans `appsettings.json` de l'application native, renseigner la chaîne de connexion Azure SQL (nom du serveur, base `gestion_licences`, identifiants et paramètres requis par Azure).

### **Construction et lancement des services conteneurisés**

Une fois Docker Desktop installé, se placer dans le dossier racine et exécuter :

- `docker compose down` pour nettoyer d'éventuels anciens conteneurs.
- `docker compose up -d --build` pour construire les images `auth-service` et `web-app`, démarrer PostgreSQL, l'API Flask et l'application web.

Après cette étape, les services sont disponibles sur la machine hôte aux adresses :

- `http://localhost:3000` pour le front web.
- `http://localhost:5000/auth` pour l'API d'authentification.
- `localhost:5432` pour un accès direct à PostgreSQL si besoin d'inspections.

## Initialisation et tests

L'initialisation de la base authdb est réalisée automatiquement par PostgreSQL au démarrage grâce au script `init.sql`. L'API Flask peut ensuite être testée à l'aide d'un client HTTP (avec PowerShell) en envoyant des requêtes POST `/auth` avec des identifiants présents dans la table `users`. L'application web est testée en ouvrant `http://localhost:3000`, en se connectant avec un compte valide, puis en vérifiant que l'utilisateur est redirigé vers `/download` et que le téléchargement de `setupLicencesBoualem.exe` se déclenche correctement.

En parallèle, la base Azure SQL est préparée via un script T-SQL exécuté dans le portail Azure ou via un outil comme Azure Data Studio. Ce script crée la base, la table `Licences` et insère les données fournies. Une fois cette étape réalisée, l'application native .NET, configurée avec une `SqlConnection` et la bonne chaîne de connexion dans `appsettings.json`, est capable de se connecter à Azure SQL et de manipuler les données de licences.

## Arrêt et nettoyage

Pour arrêter tous les services conteneurisés, il suffit d'exécuter à nouveau `docker compose down` dans le dossier racine. Cela supprime les conteneurs tout en conservant les images. Le redéploiement ultérieur se fait simplement via `docker compose up -d`.

## Conclusion

Le déploiement réalisé repose sur une séparation claire des responsabilités : un front web, un service d'authentification, une base d'authentification en PostgreSQL et une base de licences sur Azure SQL. Les choix techniques (Docker, Docker Compose, Flask, Node/Express, PostgreSQL en conteneur, Azure SQL pour la partie licences, adaptation de l'application native de MySQL vers SQL Server) ont été motivés par la volonté de respecter les contraintes du cours, de rester proche des pratiques en entreprise (services séparés, base managée dans le cloud) et d'obtenir un environnement facile à reproduire. La seule modification du code métier a consisté à adapter les informations de connexion et le type de connexion dans l'application native, afin de supporter SQL Server au lieu de MySQL, tout en gardant la logique fonctionnelle inchangée.